

# Enterprise C# and .NET Software Modernization Framework: A Multi-Cloud, Ecosystem-Agnostic Curriculum Strategy

## Architectural Imperatives for Modernizing Legacy .NET Workloads

The modernization of enterprise software assets represents a fundamental shift from monolithic, infrastructure-bound architectures to modular, cloud-native execution models<sup>1</sup>. Historically, enterprise C# and .NET applications were constructed as single-unit deployment monoliths hosted on Microsoft Internet Information Services (IIS) and Windows Server environments<sup>1</sup>. While these monoliths provided initial development simplicity, they frequently evolved into tightly coupled systems characterized by high maintenance overhead, fragile deployment pipelines, and severe horizontal scaling bottlenecks<sup>1</sup>. Modern application engineering demands that these legacy assets be refactored or rearchitected into lightweight, cross-platform components utilizing .NET 8 or modern ASP.NET Core execution models running on containerized Linux infrastructure<sup>1</sup>.

Architectural modernization requires a structured strategy to navigate the spectrum of transformation options, commonly categorized as rehosting, replatforming, refactoring, rearchitecting, and rebuilding<sup>2</sup>. Rehosting, or lift-and-shift, relocates existing virtual machines without structural changes, which preserves underlying technical debt<sup>2</sup>. In contrast, refactoring and rearchitecting break down monoliths into bounded contexts, microservices, or event-driven serverless functions<sup>2</sup>. This transition unlocks measurable operational gains, including smaller memory footprints, zero-downtime rolling updates, and sustained cloud infrastructure cost reductions often exceeding 30% to 50%<sup>1</sup>.

Organizations executing modernization programs face vendor-specific platform traps if their training curricula focus exclusively on proprietary toolchains<sup>3</sup>. Expanding educational models to incorporate alternative cloud providers—such as Amazon Web Services (AWS) and Google Cloud Platform (GCP)—alongside open-source frameworks ensures engineering organizations remain agile, cost-effective, and adaptable to heterogeneous cloud environments<sup>2</sup>. The operational mapping across these major ecosystem paradigms highlights how single-vendor Microsoft tools map directly to industry-standard multi-cloud alternatives.

| Modernization Dimension | Traditional Microsoft-Ce | AWS Cloud-Native | GCP Cloud-Native | Open-Source & |
|-------------------------|--------------------------|------------------|------------------|---------------|
|-------------------------|--------------------------|------------------|------------------|---------------|

|                                   | <b>ntrinsic Pattern</b>                                | <b>Equivalents</b>                                     | <b>Equivalents</b>                                      | <b>Ecosystem-Agnostic Alternatives</b>                               |
|-----------------------------------|--------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------------|
| Work Tracking & Collaboration     | Azure DevOps (ADO) Boards <sup>3</sup>                 | AWS Management Console / Jira Integration <sup>7</sup> | Google Cloud Issue Trackers / Jira <sup>7</sup>         | Jira Cloud + Slack <sup>7</sup> , GitLab Issues + Slack <sup>9</sup> |
| AI Code Generation                | GitHub Copilot <sup>3</sup>                            | AWS Transform / Q Developer <sup>2</sup>               | Google Gemini Code Assist <sup>12</sup>                 | Tabnine Enterprise (Air-Gapped / On-Prem) <sup>13</sup>              |
| IDE & Developer Environment       | Visual Studio 2022 <sup>3</sup>                        | VS Code with AWS Toolkit <sup>15</sup>                 | VS Code with Cloud Code <sup>12</sup>                   | JetBrains Rider (Cross-Platform) <sup>16</sup>                       |
| Automated Quality Assurance       | Microsoft Test Manager / Azure Test Plans <sup>3</sup> | AWS DevOps Test Integrations <sup>4</sup>              | GCP Automated Testing Pipelines <sup>18</sup>           | Playwright for .NET <sup>19</sup> , Tricentis Tosca <sup>3</sup>     |
| Container Runtimes                | Docker Desktop / Windows Containers <sup>3</sup>       | Docker Engine / Amazon Linux 2 Runtimes <sup>21</sup>  | Container-Optimized OS / Docker <sup>5</sup>            | Podman (Daemonless / Rootless) <sup>3</sup>                          |
| Container Orchestration           | Azure Kubernetes Service (AKS) <sup>3</sup>            | Amazon EKS / AWS App Runner <sup>4</sup>               | Google Kubernetes Engine (GKE) / Cloud Run <sup>5</sup> | Red Hat OpenShift <sup>3</sup>                                       |
| Continuous Integration & Delivery | Azure Pipelines / GitHub Actions <sup>3</sup>          | AWS CodePipeline / CloudFormation <sup>4</sup>         | Google Cloud Deploy <sup>5</sup>                        | GitLab CI/CD + ArgoCD <sup>6</sup>                                   |
| Database Persistence              | SQL Server on IaaS / Azure SQL DB <sup>3</sup>         | Amazon RDS for SQL Server / Aurora <sup>2</sup>        | Cloud SQL for SQL Server <sup>23</sup>                  | PostgreSQL / Containerized SQL Server <sup>3</sup>                   |

|                           |                                                   |                                             |                                          |                                                             |
|---------------------------|---------------------------------------------------|---------------------------------------------|------------------------------------------|-------------------------------------------------------------|
| Application Observability | Azure Monitor / Application Insights <sup>3</sup> | Amazon CloudWatch / AWS X-Ray <sup>21</sup> | Google Cloud Observability <sup>23</sup> | OpenTelemetry + Datadog / Prometheus / Grafana <sup>6</sup> |
|---------------------------|---------------------------------------------------|---------------------------------------------|------------------------------------------|-------------------------------------------------------------|

## Expanding the Curriculum Across Toolchain Layers

### Agile Work Tracking and Developer Collaboration

While Azure DevOps (ADO) Boards provide native work item tracking for Microsoft-centric shops, global engineering teams heavily utilize alternative ecosystems centered around Jira Cloud, GitLab, and Slack<sup>3</sup>. Modern developer workflows emphasize ChatOps, where project planning, pull request reviews, and build notifications are embedded directly into collaboration channels<sup>8</sup>.

Integrations such as the Jira Cloud for Slack app allow engineers to create, assign, transition, and preview work items using inline slash commands like `/jira` without leaving their primary chat interface<sup>7</sup>. Similarly, the GitLab for Slack integration leverages native slash commands (`/gitlab`), automated webhook notifications, and AI-driven bots like GitLab Duo<sup>9</sup>. GitLab Duo reads conversation context in Slack threads to automatically draft issues, summarize technical discussions, and execute CI/CD pipeline triggers<sup>9</sup>. Incorporating these tools into a modernization curriculum equips developers to manage sprint backlogs and trace code commits within multi-vendor enterprise environments<sup>3</sup>.

### AI-Assisted C# Code Generation and Enterprise Context

AI-assisted engineering has progressed beyond basic code completion to context-aware codebase transformations<sup>3</sup>. While GitHub Copilot offers standard integration across Visual Studio and VS Code, enterprise deployments often demand strict data privacy, local codebase indexing, and air-gapped security compliance<sup>3</sup>.

Tabnine provides an enterprise-grade AI coding platform that deploys on-premises or within private cloud VPCs, preventing IP exposure<sup>13</sup>. Powered by an Enterprise Context Engine, Tabnine indexes localized repositories to deliver contextually relevant C# code generation, minimal API controller refactoring, and unit test generation aligned with organizational security rules<sup>13</sup>. It broadens developer access by supporting JetBrains Rider, Visual Studio 2022, VS Code, and Eclipse<sup>17</sup>.

In the AWS ecosystem, AI guidance is delivered through agentic services like AWS Transform and the AWS Toolkit for .NET Refactoring, which evaluate legacy codebases and automate the replacement of deprecated NuGet packages<sup>11</sup>. For GCP-centric teams, Google Gemini Code Assist provides inline C# coding support tuned for Google Cloud services and SDKs<sup>12</sup>. Training curriculum must address these varied AI options to teach effective prompting, code governance, and automated test generation across platforms<sup>3</sup>.

### Integrated Development Environments: Visual Studio vs. JetBrains

## Rider

Limiting C# development instruction to Microsoft Visual Studio ignores the widespread adoption of JetBrains Rider within non-Microsoft-centric engineering teams<sup>3</sup>. JetBrains Rider is a cross-platform .NET IDE that operates natively across Windows, macOS, and Linux<sup>16</sup>. Rider provides fast code analysis, predictive debugging, built-in database tools, deep Entity Framework Core introspection, and integrated container management<sup>16</sup>. Furthermore, Rider offloads resource-intensive solution indexing to a separate background process, eliminating UI freezes commonly experienced in monolithic IDEs during large solution refactoring<sup>16</sup>. Rider offers native support for multi-agent AI assistants (including Tabnine, GitHub Copilot, and Claude) via the Agent Control Protocol (ACP)<sup>16</sup>. Demonstrating modern .NET development in both Visual Studio 2022 and JetBrains Rider ensures developers can work seamlessly on any operating system<sup>3</sup>.

## Quality Assurance: Model-Based Testing vs. Programmatic Automation

Modernization requires robust test automation to prevent regression errors when refactoring monolithic codebases<sup>2</sup>. Traditional Microsoft frameworks often relied on Microsoft Test Manager and Azure Test Plans for test management<sup>3</sup>. The curriculum updates this by contrasting model-based enterprise platforms with programmatic developer-centric testing frameworks<sup>3</sup>.

Tricentis Tosca provides a model-based test automation (MBT) approach that scans user interfaces and REST API endpoints to build reusable test modules without extensive scripting<sup>3</sup>. Tosca integrates into Azure DevOps and GitHub Actions CI/CD pipelines to execute automated regression suites and report risk-based coverage metrics<sup>3</sup>.

For code-first automation, Microsoft Playwright for .NET offers an open-source browser automation framework for asynchronous end-to-end web testing in C#<sup>19</sup>. Operating natively across Chromium, Firefox, and WebKit, Playwright integrates directly with unit testing runners such as xUnit, NUnit, and MSTest<sup>19</sup>. Teaching both Tricentis Tosca and Playwright for .NET equips teams to balance non-programmer quality assurance with developer-driven, code-first test suites<sup>3</sup>.

## Containerization Runtimes: Docker Engine and Podman

Containerization is fundamental to application modernization because it isolates application runtimes from underlying OS dependencies<sup>1</sup>. While Docker remains the historical standard, enterprise environments increasingly adopt Podman as an alternative<sup>3</sup>.

Podman provides a daemonless, rootless container architecture that mitigates the security risks associated with Docker's root-privileged daemon<sup>3</sup>. Podman uses standard OCI container images and supports identical command-line syntax, allowing developers to build, tag, and manage Linux and Windows containers interchangeably<sup>3</sup>. Multi-container application stacks can be orchestrated locally using Docker Compose or Podman Compose before pushing images to enterprise registries like Azure Container Registry (ACR), Amazon Elastic Container

Registry (ECR), or Google Artifact Registry<sup>3</sup>.

## Container Orchestration Across Cloud Providers

Once containerized, enterprise workloads require orchestrators capable of managing auto-scaling, service discovery, zero-downtime deployments, and storage attachment<sup>3</sup>.

Modernization training spans major enterprise orchestrators:

- **Red Hat OpenShift:** An enterprise Kubernetes platform offering security hardening, integrated developer consoles, and custom resources including Routes, DeploymentConfigs, and ImageStreams<sup>3</sup>. OpenShift streamlines multi-tenant application isolation and enterprise deployment pipelines<sup>3</sup>.
- **Amazon Elastic Kubernetes Service (EKS) and AWS App Runner:** Amazon EKS delivers managed Kubernetes clusters on AWS infrastructure<sup>4</sup>. For simplified serverless deployment, AWS App Runner automatically builds and deploys containerized web applications directly from ECR or source repositories, providing built-in load balancing, auto-scaling, and TLS termination without manual infrastructure management<sup>15</sup>.
- **Google Kubernetes Engine (GKE) and Google Cloud Run:** GKE offers managed Kubernetes with deep automation for cluster upgrades and scaling<sup>5</sup>. Google Cloud Run provides a serverless execution model that runs containerized C# services on a pay-per-use model, automatically scaling instances down to zero when idle<sup>5</sup>.
- **Azure App Service and Azure Kubernetes Service (AKS):** Fully managed Azure platforms for running containerized web applications and orchestrated microservice clusters<sup>3</sup>.

## CI/CD Pipelines and GitOps Automation

Automating the movement of code from commit to production requires modern CI/CD engines and GitOps practices<sup>3</sup>. While GitHub Actions provides robust native workflow execution using YAML definitions, GitLab CI/CD is widely deployed across enterprise data centers<sup>3</sup>.

A modern .gitlab-ci.yml pipeline defines structured stages to compile .NET code, run Playwright unit and integration tests, construct Docker/Podman images, and push binaries to registries<sup>6</sup>.

For continuous deployment to Kubernetes clusters, GitOps tooling like ArgoCD monitors application repositories and automatically synchronizes cluster configurations to match target manifests<sup>6</sup>. Webhooks notify developer channels in Slack via apps like Axolo, giving engineering teams real-time visibility into pipeline successes or build failures<sup>24</sup>.

## Expanded Multi-Cloud Modernization Curriculum Syllabus

### Day 1: Modern Development Foundations, Polyglot Tooling, and AI-Assisted Coding

#### Module 1: Software Modernization Strategy & Ecosystem Overview

This module establishes the core drivers and strategic pathways for converting legacy

monolithic systems into cloud-native architectures across cloud platforms<sup>1</sup>.

- Understanding the technical and financial imperatives for modernizing legacy .NET Framework and IIS applications to cross-platform .NET 8<sup>1</sup>.
- Comparative evaluation of modernization strategies: Rehosting, Replatforming, Refactoring, Rearchitecting, Rebuilding, and Replacing<sup>2</sup>.
- Automated assessment and transformation tooling: Azure Migrate, GitHub Copilot modernization extensions, AWS Toolkit for .NET Refactoring, AWS App2Container (A2C), AWS Transform, and Google Cloud assessment frameworks<sup>2</sup>.
- *Hands-on Lab*: Analyze a reference monolithic C# application; execute assessment scans using AWS Toolkit for .NET Refactoring and Azure Migrate to identify refactoring targets<sup>3</sup>.

## **Module 2: Agile Work Tracking, Sprint Planning, and ChatOps Integrations**

This module covers modern project management and ChatOps workflows that integrate planning tools directly into developer chat platforms<sup>3</sup>.

- Comparative review of agile work tracking platforms: Azure DevOps Boards, Jira Cloud, and GitLab Issues<sup>3</sup>.
- Implementing ChatOps collaboration channels:
  - Configuring the Jira Cloud for Slack app to create, transition, and search work items using /jira commands<sup>7</sup>.
  - Configuring GitLab for Slack with slash commands (/gitlab) and automated webhook event notifications<sup>9</sup>.
  - Leveraging AI bots in chat (GitLab Duo) to summarize discussions and draft issues directly within Slack threads<sup>9</sup>.
- *Hands-on Lab*: Setup a Jira/GitLab project connected to a Slack channel; perform sprint planning, execute issue updates via Slack commands, and link code commits to work items<sup>3</sup>.

## **Module 3: AI-Assisted C# Engineering: Copilot, Tabnine, and Gemini**

This module provides practical skills in leveraging enterprise AI assistants to accelerate code refactoring and test generation<sup>3</sup>.

- Architectural comparison of enterprise AI platforms: GitHub Copilot, Tabnine Enterprise, and Google Gemini Code Assist<sup>3</sup>.
- Deep dive into Tabnine Enterprise: Private context engine, local repository indexing, on-premises/air-gapped deployment, and zero-data-retention compliance<sup>13</sup>.
- Effective prompting techniques for C# / .NET 8 refactoring:
  - Converting legacy synchronous methods into non-blocking async/await code<sup>3</sup>.
  - Refactoring bloated controllers into ASP.NET Core Minimal APIs<sup>3</sup>.
  - Generating unit tests and mock objects for existing legacy business logic<sup>3</sup>.
- *Hands-on Lab*: Refactor a legacy C# module into a .NET 8 Minimal API and generate unit test suites using Tabnine in JetBrains Rider or Copilot in VS Code<sup>3</sup>.

## **Module 4: Modern .NET Architecture & IDE Ecosystem (Visual Studio & Rider)**

This module focuses on writing modern, performant C# applications using modern .NET patterns across Visual Studio 2022 and JetBrains Rider<sup>1</sup>.

- Building cloud-native REST APIs using .NET 8 Minimal APIs, ASP.NET Core, and dependency injection<sup>1</sup>.
- Selecting developer IDEs: Visual Studio 2022 vs. JetBrains Rider vs. VS Code<sup>3</sup>.
- Cross-platform development with JetBrains Rider:
  - Running natively on macOS, Linux, and Windows platforms<sup>16</sup>.
  - Advanced code analysis, performance profiling, and Entity Framework Core introspection<sup>16</sup>.
  - Multi-agent AI integration via the Agent Control Protocol (ACP)<sup>16</sup>.
- Data access strategies: Entity Framework Core vs. Dapper micro-ORM patterns for high-throughput queries<sup>3</sup>.
- Secrets management integration: Azure Key Vault, AWS Secrets Manager, and Google Secret Manager references<sup>3</sup>.
- *Hands-on Lab*: Implement a high-performance REST endpoint in .NET 8 using JetBrains Rider, execute database operations via EF Core/Dapper, and configure cloud secret bindings<sup>3</sup>.

## **Module 5: Automated Quality Assurance: Tricentis Tosca & Playwright for .NET**

This module covers automated quality assurance strategies combining model-based testing with programmatic browser automation<sup>3</sup>.

- Evaluating QA paradigms: Model-Based Testing (MBT) vs. Developer-Centric Programmatic Automation<sup>3</sup>.
- Model-based UI and API automation using Tricentis Tosca: Module scanning, Execution Lists, and CI/CD reporting<sup>3</sup>.
- End-to-end web test automation with Microsoft Playwright for .NET:
  - Constructing multi-browser end-to-end tests in C# (Chromium, Firefox, WebKit)<sup>19</sup>.
  - Integrating Playwright suites with xUnit, NUnit, and MSTest runners<sup>19</sup>.
  - Headless browser execution inside containerized deployment pipelines<sup>19</sup>.
- *Hands-on Lab*: Build an automated API test project in Tricentis Tosca and draft a C# end-to-end web test using Playwright for .NET; execute both in automated test runners<sup>3</sup>.

## **Day 2: Containerization, Multi-Cloud Orchestration, CI/CD, and DevOps Operationalization**

### **Module 6: Container Runtimes & Packaging: Docker Engine and Podman**

This module details containerization mechanics for .NET workloads using industry-standard container engines<sup>3</sup>.

- Container architecture principles: OCI specifications, layers, images, and runtimes<sup>3</sup>.
- Engine comparison: Docker Engine vs. Podman (daemonless, rootless container execution)<sup>3</sup>.
- Writing optimized multi-stage Dockerfiles for .NET 8 applications to minimize image size

and attack surface<sup>3</sup>.

- Managing enterprise container registries: Azure Container Registry (ACR), Amazon ECR, and Google Artifact Registry<sup>3</sup>.
- Local multi-container testing using Docker Compose and Podman Compose<sup>3</sup>.
- *Hands-on Lab*: Package a .NET 8 REST service into a container using Podman/Docker, run a multi-container stack with Compose, and push the image to an enterprise registry<sup>3</sup>.

## **Module 7: Multi-Cloud Container Orchestration: OpenShift, EKS, GKE, and Serverless Containers**

This module demonstrates container deployment and orchestration across major enterprise cloud platforms<sup>3</sup>.

- Kubernetes orchestration core components: Pods, Deployments, Services, Ingress, ConfigMaps, and Secrets<sup>3</sup>.
- Multi-cloud orchestration options:
  - *Red Hat OpenShift*: Routes, DeploymentConfigs, ImageStreams, and security constraints<sup>3</sup>.
  - *AWS Orchestration*: Managing clusters on Amazon EKS vs. serverless execution on AWS App Runner<sup>4</sup>.
  - *Google Cloud Orchestration*: Deploying to Google Kubernetes Engine (GKE) vs. serverless containers on Cloud Run<sup>5</sup>.
- Managing stateful workloads and persistent storage for SQL database engines<sup>2</sup>.
- *Hands-on Lab*: Deploy the containerized .NET application to Red Hat OpenShift or Amazon EKS; configure routing, liveness/readiness probes, and horizontal pod autoscaling<sup>3</sup>.

## **Module 8: Enterprise CI/CD Pipelines, GitOps, and Slack Notifications**

This module constructs end-to-end automated build and deployment pipelines integrated with ChatOps feedback channels<sup>3</sup>.

- CI/CD pipeline automation engines: GitHub Actions, GitLab CI/CD, and AWS CodePipeline<sup>3</sup>.
- Building multi-stage pipelines in GitLab CI/CD (.gitlab-ci.yml) and GitHub Actions:
  - Automated compilation and testing (dotnet build, dotnet test, Playwright)<sup>3</sup>.
  - Container image construction, vulnerability scanning, and registry pushing<sup>3</sup>.
  - Automated deployment triggers for Kubernetes, AWS App Runner, or GCP Cloud Run<sup>5</sup>.
- Implementing GitOps deployment workflows using ArgoCD<sup>6</sup>.
- Integrating ChatOps notifications: Sending real-time pipeline status updates and test failures to Slack using Axolo and custom webhooks<sup>24</sup>.
- *Hands-on Lab*: Build a complete CI/CD pipeline using GitLab CI/CD or GitHub Actions that compiles code, executes Playwright tests, builds a container, deploys via GitOps/App Runner, and sends alerts to Slack<sup>3</sup>.

## **Module 9: Multi-Cloud Deployment, Governance, and Observability**

This module addresses operational governance, cloud service selection, and unified telemetry across modern deployment targets<sup>3</sup>.

- Selecting cloud hosting models for modernized workloads:
  - *Azure*: Azure App Service, Azure Container Apps, AKS<sup>3</sup>.
  - *AWS*: AWS App Runner, Amazon ECS, Amazon EKS<sup>4</sup>.
  - *GCP*: Google Cloud Run, Google GKE<sup>5</sup>.
- Application observability and monitoring standards:
  - Instrumenting .NET applications with OpenTelemetry standards<sup>6</sup>.
  - Routing telemetry to Azure Monitor, Amazon CloudWatch, and Google Cloud Observability<sup>3</sup>.
- Infrastructure as Code (IaC) principles using Terraform, Bicep, and AWS CloudFormation<sup>3</sup>.
- *Hands-on Lab*: Deploy the application stack to AWS App Runner or GCP Cloud Run; configure OpenTelemetry instrumentation and analyze traces in CloudWatch or Google Cloud Observability<sup>3</sup>.

## Module 10: Capstone Project: End-to-End Enterprise Modernization

This capstone module brings together all tools, frameworks, and deployment targets covered in the course within a practical team scenario<sup>3</sup>.

- *Team Execution Pipeline*:
  1. Manage tasks and pull requests in Jira Cloud or GitLab Issues integrated with Slack<sup>3</sup>.
  2. Refactor legacy C# modules into .NET 8 Minimal APIs using JetBrains Rider with Tabnine/Copilot AI assistance<sup>3</sup>.
  3. Containerize the application into an OCI image using Podman/Docker<sup>3</sup>.
  4. Run automated C# end-to-end tests using Playwright for .NET and API tests using Tricentis Tosca<sup>3</sup>.
  5. Trigger continuous deployment via GitLab CI/CD or GitHub Actions to Red Hat OpenShift, AWS EKS/App Runner, or GCP Cloud Run<sup>3</sup>.
  6. Validate deployment notifications in Slack via Axolo webhooks and verify application telemetry via OpenTelemetry<sup>3</sup>.
- *Debrief*: Review modern architecture decisions, pipeline performance metrics, and multi-cloud governance strategies<sup>3</sup>.

## Strategic Implementation, Governance, and Telemetry Integration

Transitioning an enterprise software modernization curriculum from a single-vendor model to a multi-cloud framework introduces critical governance, security, and operational considerations<sup>2</sup>. Engineering leadership must establish guardrails across three key areas:

### AI Context Governance and Data Privacy

Deploying AI coding assistants like GitHub Copilot, Tabnine, or Google Gemini requires clear

rules regarding intellectual property and data isolation<sup>3</sup>. Organizations operating under strict regulatory constraints (such as financial or healthcare environments) should prioritize AI platforms that offer local codebase indexing, zero data retention, and air-gapped deployment options<sup>13</sup>. Tabnine's private context engine allows teams to leverage localized repository training without transmitting proprietary source code to public cloud APIs<sup>13</sup>.

| AI Coding Platform | Deployment Architecture                               | Local Repository Indexing                       | Data Privacy & Retention Policy                      | Supported IDEs                                           |
|--------------------|-------------------------------------------------------|-------------------------------------------------|------------------------------------------------------|----------------------------------------------------------|
| GitHub Copilot     | Multi-Tenant Cloud / SaaS <sup>3</sup>                | Cloud-Based Indexing <sup>3</sup>               | Subject to SaaS Data Policies <sup>3</sup>           | Visual Studio, VS Code, JetBrains <sup>3</sup>           |
| Tabnine Enterprise | On-Premises, Private VPC, or Air-Gapped <sup>13</sup> | Private Enterprise Context Engine <sup>13</sup> | Zero Data Retention & No Code Exposure <sup>13</sup> | JetBrains Rider, VS 2022, VS Code, Eclipse <sup>17</sup> |
| Gemini Code Assist | Managed Google Cloud Tenant <sup>12</sup>             | Google Cloud Workspace Indexing <sup>12</sup>   | Protected under GCP Enterprise Terms <sup>12</sup>   | VS Code, JetBrains IDEs <sup>12</sup>                    |

## Container Security and Rootless Operations

Shifting runtimes to Linux containers and Podman/Docker requires automated vulnerability management within build pipelines<sup>3</sup>. CI/CD pipelines must incorporate static container image scanning (utilizing tools such as Trivy, Clair, or native registry scanners in ECR, ACR, and Artifact Registry) prior to pushing images to target orchestrators<sup>3</sup>. Furthermore, executing containers in rootless mode via Podman reduces host-level compromise risks in multi-tenant environments like Kubernetes and OpenShift<sup>3</sup>.

## Unified Telemetry via OpenTelemetry

Avoiding cloud vendor lock-in at the application layer requires standardizing telemetry collection<sup>6</sup>. Rather than embedding vendor-specific SDKs (such as proprietary Azure Application Insights or AWS X-Ray libraries) directly into C# code, applications should incorporate OpenTelemetry APIs<sup>3</sup>. OpenTelemetry provides a vendor-neutral standard for emitting metrics, logs, and distributed traces<sup>6</sup>. Collectors then route this telemetry to any designated monitoring backend—whether Azure Monitor, Amazon CloudWatch, Google Cloud Observability, Datadog, or Grafana—without requiring application code changes<sup>3</sup>.

# Conclusions

Modernizing legacy C# and .NET applications requires an architectural strategy that extends beyond vendor-specific platforms<sup>1</sup>. While Microsoft-centric tools such as Azure DevOps, Visual Studio 2022, and Azure App Service remain viable, enterprise development teams operate within multi-cloud environments requiring flexible, ecosystem-agnostic skillsets<sup>3</sup>.

Integrating non-Microsoft tools into the modernization curriculum—including planning with Jira and GitLab via Slack ChatOps, AI assistance through Tabnine Enterprise, cross-platform IDE development using JetBrains Rider, code-first test automation with Playwright for .NET, rootless containerization via Podman, multi-cloud deployment targeting AWS App Runner, EKS, and Google Cloud Run, and CI/CD automation with GitLab CI/CD and ArgoCD—prepares engineering organizations to execute modernization strategies across heterogeneous technology stacks<sup>3</sup>. This approach reduces cloud vendor lock-in, improves developer productivity across platforms, and ensures long-term operational resilience<sup>1</sup>.

## Works cited

1. Architecting-Modern-Web-Applications-with-ASP.NET-Core-and-Azure.pdf
2. The complete guide to AWS application modernization - vFunction, <https://vfunction.com/blog/aws-application-modernization-guide/>
3. Software Development Modernization.docx
4. Modernising in AWS: Migrating from .NET to Containerised .NET Core | Devoteam, <https://www.devoteam.com/expert-view/modernising-in-aws-migrating-from-net-to-containerised-net-core/>
5. Cloud Deploy - Fully Managed Continuous Delivery | Google Cloud, <https://cloud.google.com/deploy>
6. Deploying a Scalable Full-Stack Application with Docker, Kubernetes, Argo CD, and GitLab CI: A Step-by-Step Guide | by Ramzi Bouzaiene | DevOps.dev, <https://blog.devops.dev/deploying-a-scalable-full-stack-application-with-docker-kubernetes-argo-cd-and-gitlab-ci-a-a471c56a3ac3>
7. Configure the Slack integration | Jira Product Discovery - Atlassian Support, <https://support.atlassian.com/jira-product-discovery/docs/configure-the-slack-integration/>
8. An easier way to use JIRA with Slack, <https://slack.com/blog/productivity/an-easier-way-to-use-jira-with-slack>
9. GitLab for Slack app, [https://docs.gitlab.com/user/project/integrations/gitlab\\_slack\\_application/](https://docs.gitlab.com/user/project/integrations/gitlab_slack_application/)
10. GitLab & Slack Integration | Slack Marketplace, <https://slack.com/marketplace/A676ADMV5-gitlab>
11. Deploy application containers to Amazon EKS with AWS App2Container, <https://docs.aws.amazon.com/app2container/latest/UserGuide/a2c-integrations-eks.html>
12. Gemini Code Assist for teams and businesses, <https://codeassist.google/products/business>

13. Tabnine Docs: Overview, <https://docs.tabnine.com/main>
14. Tabnine AI Code Assistant | Smarter AI Coding Agents. Total Enterprise Control., <https://www.tabnine.com/>
15. Modernize your .NET application using AWS App2Container and AWS Toolkit for .NET, <https://aws.amazon.com/blogs/containers/replatform-and-refactor-your-workload-to-aws-using-aws-app2container-and-aws-toolkit-for-net-refactoring-application/>
16. Rider: the cross-platform .NET and game development IDE from JetBrains, <https://www.jetbrains.com/rider/>
17. Supported IDEs - Tabnine Docs, <https://docs.tabnine.com/main/welcome/readme/supported-ides>
18. Deploy a .NET app to Google Kubernetes Engine using a GitHub Actions workflow, <https://docs.cloud.google.com/dotnet/docs/getting-started/deploying-to-gke-using-github-actions>
19. How to test web applications with Playwright and C# .NET | Twilio, <https://www.twilio.com/en-us/blog/developers/community/test-web-apps-with-playwright-and-csharp-dotnet>
20. Writing tests | Playwright .NET, <https://playwright.dev/dotnet/docs/writing-tests>
21. Modernize Java and .NET applications remotely using AWS App2Container | Containers, <https://aws.amazon.com/blogs/containers/modernize-java-and-net-applications-remotely-using-aws-app2container/>
22. Expanding modernization scenarios using the AWS App2Container integration with AWS App Runner | Containers, <https://aws.amazon.com/blogs/containers/expanding-modernization-scenarios-using-the-aws-app2container-integration-with-aws-app-runner/>
23. Quickstart: Build and deploy a .NET web app to Google Cloud with Cloud Run, <https://docs.cloud.google.com/run/docs/quickstarts/build-and-deploy/deploy-dotnet-service>
24. GitLab Action Slack Notification - Axolo, <https://axolo.co/features/gitlab-ci-cd-slack-notifications>
25. tabnine-vscode/README.md at master - GitHub, <https://github.com/codota/tabnine-vscode/blob/master/README.md?plain=1>
26. Tabnine for Visual Studio 2022 is now generally available, <https://www.tabnine.com/blog/tabnine-is-now-available-for-visual-studio-2022/>
27. Build your first end-to-end test with Playwright - Training - Microsoft Learn, <https://learn.microsoft.com/en-us/training/modules/build-with-playwright/>
28. How to set up Slack notifications from GitLab CI/CD using the Slack API - DEV Community, <https://dev.to/cheulong/how-to-set-up-slack-notifications-from-gitlab-cicd-using-the-slack-api-h5f>
29. GitLab CI/CD Integration with Slack (Part 2) | by Neha Saini - Medium, <https://medium.com/@neha-saini/gitlab-ci-cd-integration-with-slack-part-2-9f8e>

[4da98391](#)